System Programming CMPE411

Assignment report

SUBMITTED TO

Dr. Felix Babalola

By

Ahmed Mohamed 22203941

Synchronization Approach

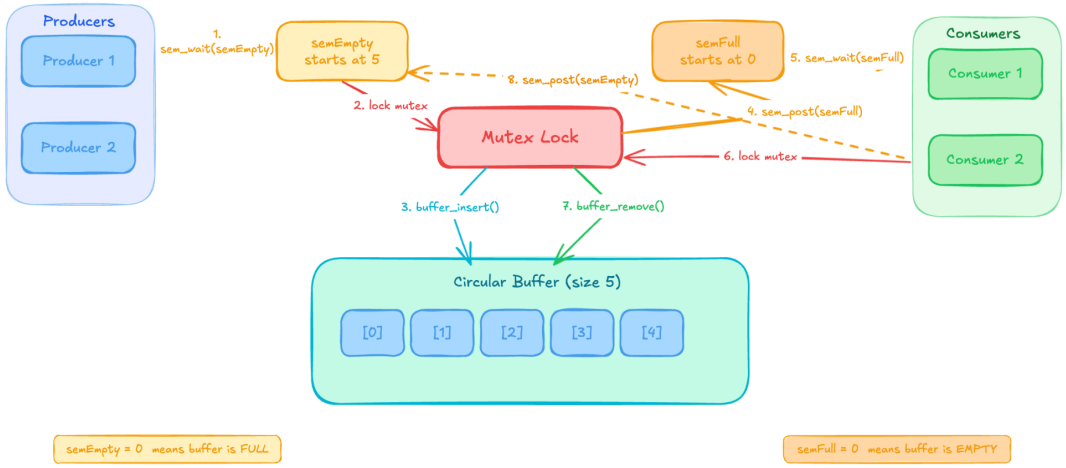
The program uses two semaphores and one mutex to keep the producers and consumers working correctly together without corrupting the buffer.

The mutex (mutext) acts as a lock that prevents two threads from touching the buffer at the same time. Whenever a thread needs to read or modify the buffer — whether inserting or removing an item — it first grabs the lock, does its work, then releases it. This guarantees that only one thread is inside the critical section at any moment.

The two semaphores handle a different problem. semEmpty tracks how many empty slots are left in the buffer, starting at 5. Before a producer inserts anything, it calls sem_wait(&semEmpty) to claim one of those slots. If the buffer is full, this blocks the producer until a consumer frees a slot. semFull works the other way — it tracks how many items are currently in the buffer, starting at 0. A consumer calls sem_wait(&semFull) before removing anything, and blocks if the buffer is empty.

After every insert, the producer calls sem_post(&semFull) to signal that a new item is ready. After every remove, the consumer calls sem_post(&semEmpty) to signal that a slot opened up. This back-and-forth keeps producers and consumers in sync without busy waiting.

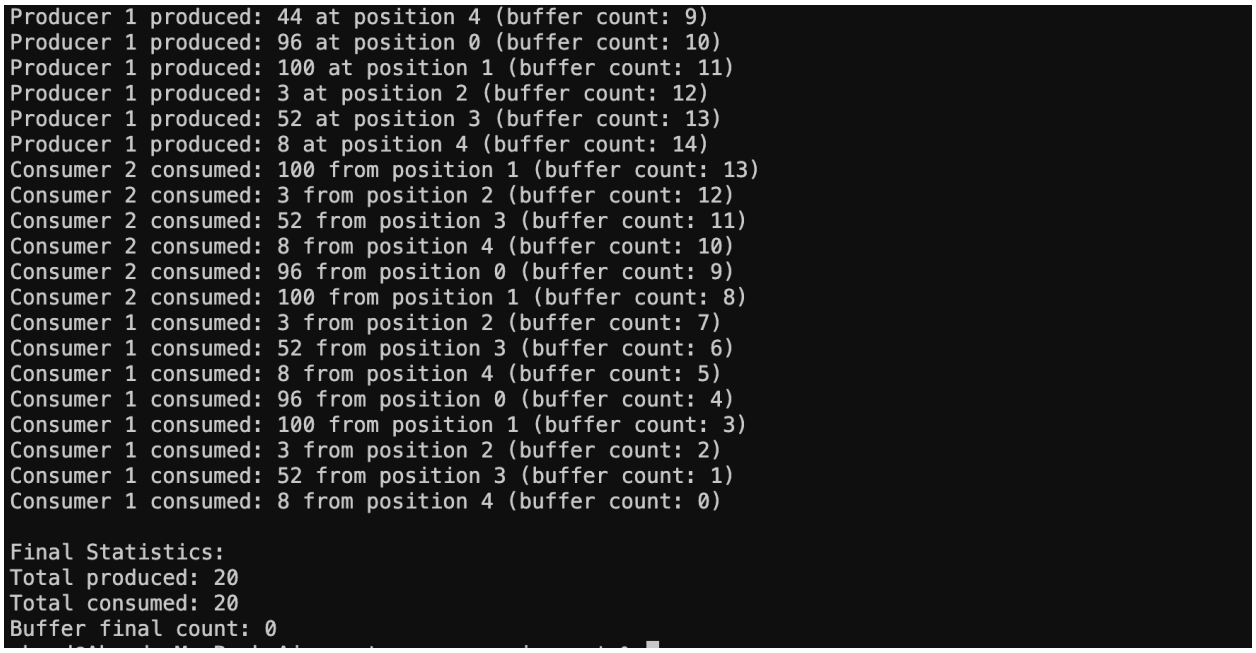
Diagram



PRODUCER FLOW:
sem_wait(semEmpty) --> lock mutex --> buffer_insert() --> unlock mutex --> sem_post(semFull)

CONSUMER FLOW:
sem_wait(semFull) --> lock mutex --> buffer_remove() --> unlock mutex --> sem_post(semEmpty)

Screenshot of Program Execution



Challenges Faced

The first challenge was getting the circular buffer wrap-around logic right. When tail or head reaches index 4, it needs to go back to 0 instead of going out of bounds. This was handled with a simple if statement checking against BUFFER_SIZE - 1.

The second challenge was the order of semaphore and mutex calls. Early on I had sem_wait inside the mutex lock which would have caused a deadlock — if the buffer is full and the producer is holding the mutex while waiting on the semaphore, the consumer can never acquire the mutex to remove an item and release the semaphore. The fix was to always call sem_wait before locking the mutex.

The third issue was saving the position before inserting or removing. Since buffer_insert() moves tail forward immediately, saving pos = tail after the call would give the wrong position. It had to be saved before the call.

What Would Happen if You Removed the Mutex?

Removing the mutex would expose the program to race conditions on any shared variable — mainly count, head, and tail.

A race condition happens when two threads access and modify a shared variable at the same time, and the final result depends on which thread happens to run first. For example, imagine both Consumer 1 and Consumer 2 check count at the same moment and both see count = 1. Both decide there is an item to consume. Consumer 1 removes it and sets count to 0. Then Consumer 2 also removes from the same position, getting garbage data, and sets count to -1. Now the buffer count is negative and the state of the buffer is completely corrupted.

The same thing can happen with tail — two producers could both read tail = 2 at the same time, both write their item to position 2 (one overwriting the other), and both increment tail to 3. One item is lost entirely.

Without the mutex there is no guarantee of atomicity — no guarantee that checking the buffer state and modifying it happens as one uninterrupted operation. The semaphores alone are not enough because they only control when threads are allowed to enter, not what happens when two threads are inside at the same time.